

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №7
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Реалізація клієнт-серверної взаємодії на основі протоколу WebSocket.

Мета роботи: Розробити клієнт-серверну архітектуру, ознайомитися з WebSocket протоколом.

Теоретичні відомості

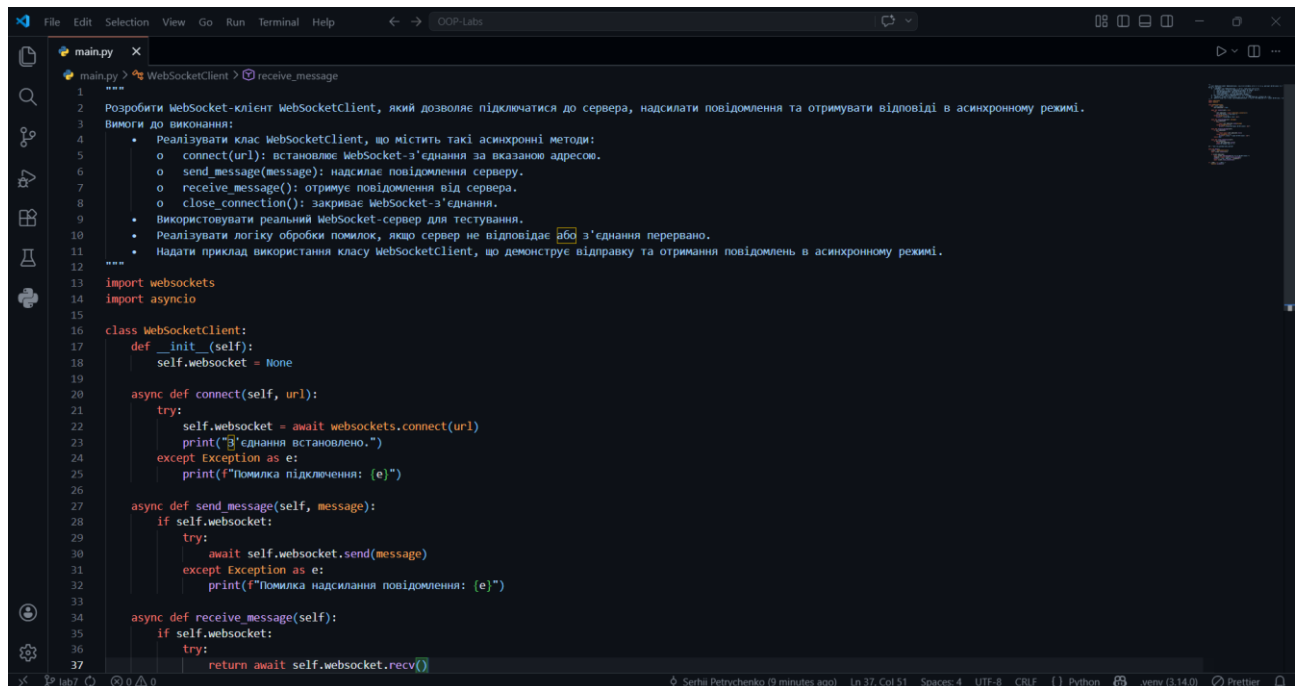
WebSocket — це протокол зв'язку, який забезпечує постійне двостороннє з'єднання між клієнтом і сервером. На відміну від стандартного HTTP, де кожен запит ініціюється клієнтом, WebSocket дозволяє серверу надсилати повідомлення клієнту без попереднього запиту.

Основні особливості та переваги протоколу:

- Постійне з'єднання: клієнт і сервер підтримують активний канал зв'язку протягом усього сеансу.
- Двостороння комунікація: обидві сторони можуть ініціювати надсилання даних у будь-який момент.
- Ефективність: протокол споживає менше мережевого трафіку та має менші затримки порівняно з HTTP.
- Сфера застосування: ідеально підходить для сервісів реального часу, таких як чати, ігри або біржові додатки.

Процес роботи починається з ініціації з'єднання через спеціальний запит (ws:// або wss://), після чого відбувається обмін повідомленнями в реальному часі до моменту закриття каналу однією зі сторін.

Результати виконаної роботи



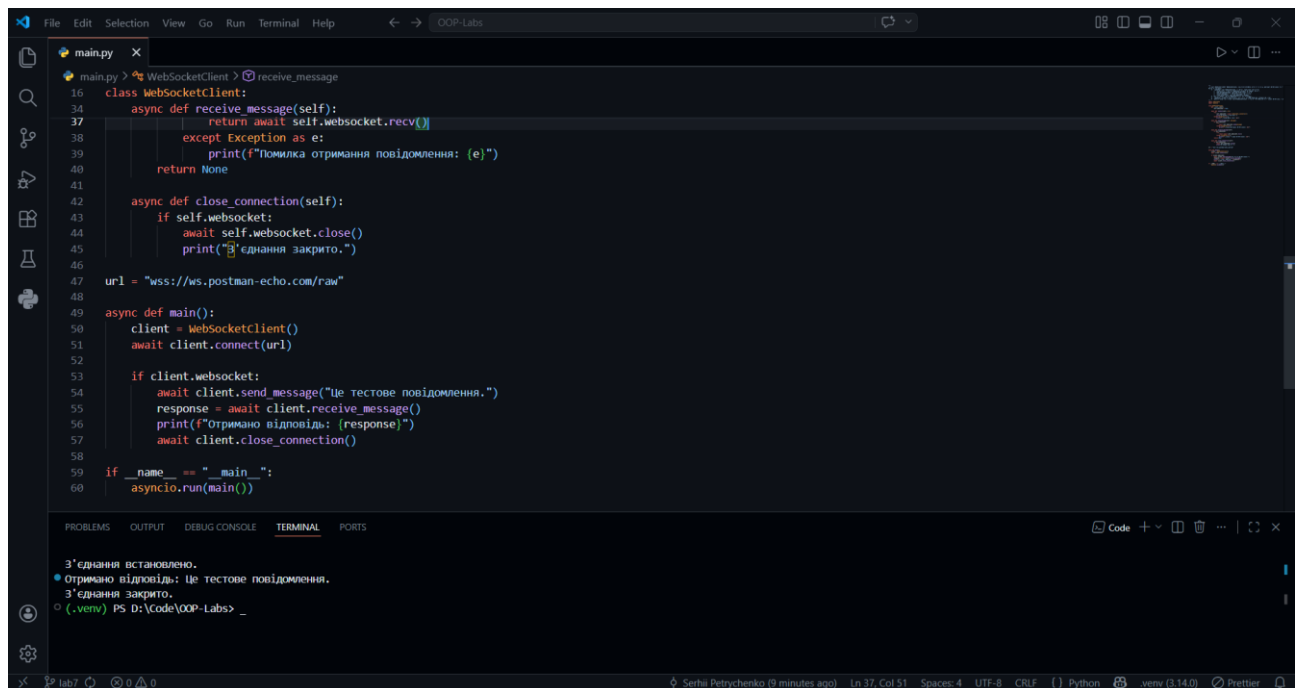
```
main.py x
main.py > WebSocketClient > receive_message
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
import websockets
import asyncio

class WebSocketClient:
    def __init__(self):
        self.websocket = None

    async def connect(self, url):
        try:
            self.websocket = await websockets.connect(url)
            print("З'єднання встановлено.")
        except Exception as e:
            print(f"Помилка підключення: {e}")

    async def send_message(self, message):
        if self.websocket:
            try:
                await self.websocket.send(message)
            except Exception as e:
                print(f"Помилка надсилання повідомлення: {e}")

    async def receive_message(self):
        if self.websocket:
            try:
                return await self.websocket.recv()
```



```
main.py x
main.py > WebsocketClient > receive_message
16 class WebsocketClient:
34     async def receive_message(self):
37         return await self.websocket.recv()
38     except Exception as e:
39         print(f"Помилка отримання повідомлення: {e}")
40     return None
41
42     async def close_connection(self):
43         if self.websocket:
44             await self.websocket.close()
45             print("З'єднання закрито.")
46
47 url = "ws://ws.postman-echo.com/raw"
48
49     async def main():
50         client = WebsocketClient()
51         await client.connect(url)
52
53         if client.websocket:
54             await client.send_message("Це тестове повідомлення.")
55             response = await client.receive_message()
56             print(f"Отримано відповідь: {response}")
57             await client.close_connection()
58
59 if __name__ == "__main__":
60     asyncio.run(main())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

З'єднання встановлено.
Отримано відповідь: Це тестове повідомлення.
З'єднання закрито.
(.venv) PS D:\code\OOP-Labs> _

Висновки

1. У ході лабораторної роботи було розроблено асинхронний клас `WebsocketClient` на мові Python з використанням бібліотеки `websockets`. Реалізовано функціонал для встановлення з'єднання, відправлення текстових повідомлень, отримання відповідей від сервера та коректного закриття каналу. Також додано механізми обробки помилок для забезпечення стабільності зв'язку.
2. Дана розробка може бути використана як база для створення систем сповіщень у реальному часі, клієнтів для онлайн-чатів або інструментів для моніторингу потокових даних (наприклад, курсів криптовалют або котирувань акцій), де критично важливо отримувати інформацію миттєво без постійного опитування сервера.
3. Робота дозволила опанувати навички асинхронного програмування в Python за допомогою модуля `asyncio`. Було вивчено відмінності між запитом "запит-відповідь" (HTTP) та постійним потоковим з'єднанням, а також отримано практичний досвід роботи з асинхронними контекстними менеджерами та обробкою винятків у мережевих з'єднаннях.
4. Переваги: висока швидкість передачі даних та можливість миттєвої реакції сервера на події. Зменшення навантаження на мережу через відсутність зайвих заголовків у кожному повідомленні.
Недоліки: необхідність постійної підтримки активного з'єднання, що споживає ресурси сервера (пам'ять). Також протокол складніший у реалізації обробки розривів зв'язку та безпеки порівняно зі звичайним REST API..